

Specifica e verifica di sistemi critici

Giulia

4 maggio 2026

1 Abstract State Machines (parte 1): stato ASM

Le ASM (Abstract State Machine) possono essere considerate FSM con stati generalizzati. Rappresentano la forma matematica di macchine virtuali ed estendono la nozione di Finite State Machine ampliando la definizione di stato e modificando la forma delle transizioni.

Si passa da stati di controllo non strutturati a **stati strutturati** che modellano:

- dati complessi arbitrari (con domini di base e funzioni per la struttura);
- operazioni per la manipolazione di dati.

Lo stato è una struttura *multi-sorted* (si possono avere domini di qualsiasi tipo) *first-order* (usa quantificatori), ovvero un'algebra del primo ordine.

Le transizioni sono **regole** che descrivono il cambiamento delle funzioni da uno stato al successivo (regola di transizione base: **if Condition then Updates**).

I costruttori di regole sono diversi, ma finiti e semplici:

- parallelismo (**par**) e azioni sequenziali (**seq**);
- iterazioni (**while**) e invocazione di sottomacchine;
- non-determinismo (**choose**) e parallelismo sincrono non limitato (**forall**);
- agenti multipli sincroni/asincroni.

Le ASM sono analoghe alle FSM; le differenze principali riguardano:

- **la concezione degli stati**: nelle FSM esiste un unico stato di controllo (*control state*), che può assumere valori in un insieme *finito* di un certo tipo, mentre nelle ASM lo stato è più complesso (algebra);
- **le condizioni di input e le azioni di output**: nelle FSM si usa un alfabeto finito, mentre nelle ASM l'input può essere una qualsiasi espressione e le azioni sono generali.

Per riassumere: gli stati di una ASM sono delle strutture algebriche, dette (semplicemente) algebre. Le transizioni permettono di modificare la struttura algebrica durante la run o esecuzione della ASM

Vocabolario

DEF. Un vocabolario (o segnatura) Σ è una collezione finita di nomi di funzioni

DEF. Fissato un vocabolario Σ , uno stato A su Σ è un insieme non vuoto X , detto superuniverso di A , con le interpretazioni dei nomi delle funzioni di Σ

- se f un nome di funzione n -aria (di n argomenti) di Σ allora la sua interpretazione f^A è una funzione di X^n a X
- Se c è un nome di costante è un elemento di X di Σ , allora la sua interpretazione C^A è un elemento di X

Le funzioni possono essere dinamiche o statiche, a seconda che l'interpretazione del nome della funzione cambia o no da uno stato al successivo

Costanti

Funzioni statiche costanti di arietà zero sono dette, ogni vocabolario ASM contiene sempre le costanti statiche *undef*, *True*, *False*. Altri esempi sono: i numeri sono costanti numeriche (l'utente può aggiungerne di sue `minimoVoto = 18`)

Funzioni statiche

Le funzioni statiche (di arietà ≥ 0) sono definite tramite una legge fissa. Esempio di funzioni statiche sono le usuali operazioni tra numeri: $+$, $-$, $\dots$ tra booleani: AND, OR, $\dots$ Sono standard o l'utente ne può definire di sue es: `max(n, m)`

Funzioni dinamiche

Cambiano la propria interpretazione al variare dello stato Funzioni dinamiche di arietà zero sono le comuni variabili dei linguaggi di programmazione (Es: `insertCoin`, `DisplayTime`)

Esempio di stato ASM Modello ASM: CLOCK real time Vocabolario (o segnatura) Σ

- Funzioni dinamiche: `CurrTime`: Real (viene incrementato dall'esterno), `DisplayTime`: Real
- Funzioni statiche: `Delta`: Real (intervallo di tempo) $+$: Real x Real $-;$ Real (addizione)

Domini ASM

Il superuniverso di uno stato ASM è "suddiviso" in universi, rappresentati dalle loro funzioni caratteristiche Se X è il superuniverso dello stato A , l'universo G di A è rappresentato dalla funzione $G : X \rightarrow Bool$ tale che $G(t) = true$ per tutti gli elementi t del superuniverso X di A che formano la partizione (o sottoinsieme) G ; $G(t) = false$, altrimenti

Esempio: $X = 1, 2, 3, a, b, mario, pippo$ ripartito in domini: $Interi = 1, 2, 3$, $Char = a, b$, $String = mario, pippo$

Termini ASM

DEF. I termini di $\sum$ sono espressioni sintattiche così costruite:

1. Variabili $v_0, v_1, v_2, \dots$ sono termini
2. Costanti c of $\sum$ sono termini
3. Se f è un nome di funzione n -aria di $\sum$ e $t_1, \dots, t_n$ sono termini, allora $f(t_1, \dots, t_n)$ è un termine.

Un termine che non contiene variabili è detto chiuso. Esempi: insertCoin, available(MILK), DisplayTime, ...

2 Abstract State Machines (parte 2): regole di transizione

In matematica le algebre sono statiche : non cambiano col passare del tempo.

In Informatica, gli stati sono dinamici : evolvono essendo aggiornati durante le computazioni.

Aggiornare stati astratti (abstract states) significa cambiare l'interpretazione delle (o solo di alcune) funzioni dinamiche della segnatura della macchina.

Il modo in cui una macchina ASM aggiorna il proprio stato è descritto da regole di transizione (transitions rules) di una certa "forma". L'insieme delle regole di transizione di una ASM definiscono la sintassi di un programma ASM Sia $\sum$ un vocabolario. Le regole di transizione di una ASM sono espressioni sintattiche su $\sum$ generate come segue attraverso l'uso di costruttori di regole

Costruttore base: update rule

Update Rule: $\mathbf{f(t_1, \dots, t_n) := s}$ dove f è un nome di funzione dinamica n -aria di $\sum$ e $t_1, \dots, t_n$ e s sono termini di $\sum$. Ossia nello stato successivo, il valore di f per gli argomenti $t_1, \dots, t_n$ è aggiornato a s . Se f è 0-aria, cioè una variabile x , l'aggiornamento ha la forma $\mathbf{x := s}$

L'update rule è il costrutto di base delle regole di transizione e determina l'aggiornamento di una locazione ASM

Locazione ASM

Nelle ASM gli stati sono associati a un insieme di valori di qualsiasi tipo, memorizzate in apposite locazioni. Matematicamente una locazione è definita come la coppia $(f, (v_1, \dots, v_n))$ con f nome di funzione e $(v_1, \dots, v_n)$.

In uno stato S , una locazione ha un valore: l'interpretazione della funzione nello stato

Funzione può essere: Variabili (0-arie) $x, y, \dots$ e Funzioni (n -arie) $f(1), \text{name}(2), \dots$

Le transizioni di stato delle ASM, tramite l'update rule, causano aggiornamenti dei valori contenuti nelle locazioni

Se $(loc = (f, (v_1, \dots, v_n)))$ ed valore $(loc) = a$, l'aggiornamento (o update) di loc ha la forma $f(v_1, \dots, v_n) := newval$, l'aggiornamento cambia il valore di loc, e quindi di $f(v_1, \dots, v_n)$, da a a $newval$

Costruttore if-then-else

Conditional Rule: **if φ then R else S** ossia se φ è vera, allora esegui R , altrimenti segui S

Classificazione delle funzioni ASM

Sia M una ASM e **env** l'ambiente di M

Dynamic: i valori dipendono dagli stati di M

- **in (monitored):** lette (non aggiornate) da M , scritte da env
- **out:** scritte (ma non lette) da M , lette da env
- **controlled:** lette e scritte da M
- **shared:** lette e scritte da M e da env

Derived: valori computati da funzioni monitorate e funzioni statiche per mezzo di una "legge" o "schema" fissati a priori

Costruttori skip e let

Skip Rule: non fare niente Let Rule (se R ha parametri, realizza il passaggio per valore): **let $x = t$ in $R(x)$** assegna il valore di t a x ed esegui R

Costruttore "par" per block rule

Block Rule (composizione parallela): R e S sono eseguite in parallelo

```
par
    R
    S
endpar
```

Aggiornamenti Consistenti

A causa del parallelismo (regole Block e Forall), una regola di transizione può richiedere più volte l'aggiornamento di una stessa funzione per gli stessi argomenti si richiede in tal caso che tali aggiornamenti siano consistenti (non posso aggiornare la stessa locazione contemporaneamente)

data la funzione $f(a_1, \dots, a_n)$ in uno stato S_i della macchina:

- La coppia $loc = (f, (a1, \dots, an))$ è detta locazione e rappresenta matematicamente il valore di $f(a1, \dots, an)$ in memoria
- Un aggiornamento (o update) è la coppia: $(loc, b) = ((f, (a1, \dots, an)), b)$ ossia l'interpretazione della funzione f in S_i viene modificata per gli argomenti $a1, \dots, an$ con il valore b in S_{i+1}
- Un update set è un insieme di aggiornamenti

DEF: Un update set U è consistente, se vale la condizione: Per ogni locazione $(f, (a1, \dots, an))$, se $((f, (a1, \dots, an)), b) \in U$ ed $((f, (a1, \dots, an)), c) \in U$, allora $b = c$

se b è diverso da c allora l'update si dice inconsistente

Se l'update set U è consistente, allora i suoi aggiornamenti possono essere effettivamente eseguiti (fired) in un dato stato.

Il risultato è un nuovo stato (di arrivo) dove le interpretazioni dei nomi delle funzioni dinamiche sono cambiati secondo U .

Le interpretazioni dei nomi delle funzioni statiche sono gli stessi dello stato precedente (di partenza).

Le interpretazioni dei nomi delle funzioni monitorate sono date dall'ambiente esterno e possono dunque cambiare in maniera arbitraria.

Rule constructors

(Macro) Call Rule: $r[t_1, \dots, t_n]$ ossia chiama r (regola con parametri) con argomenti $t_1, \dots, t_n$

Una definizione di regola per un nome di regola r di arietà n è un'espressione $r(x_1, \dots, x_n) = R$ dove R è una regola di transizione. In una call rule $r[t_1, \dots, t_n]$ le variabili x_i che occorrono nel corpo R della definizione di r vengono sostituite dai parametri t_i (modularità: perché puoi definire la regola una volta e richiamarla più volte con argomenti diversi, senza riscriverla ogni volta) Il risultato è che nel corpo RR ogni occorrenza di x_i viene rimpiazzata da t_i : $R[x_1 \rightarrow t_1, x_2 \rightarrow t_2, \dots, x_n \rightarrow t_n]$

Costruttore seq

Seq Rule (composizione sequenziale): **seq** R S **endseq** Significato: R e S sono eseguite in sequenza (gli update causati in R sono già visibili in S ; lo stato tra R ed S non è visibile)

Costruttori forall e choose

Forall Rule: **forall** x **with** φ **do** $R[x]$ Significato: Esegui R in parallelo per ogni x che soddisfa φ Implementa il concetto di parallelismo sincrono (potentially unbounded)

Choose Rule: **choose** x **with** φ **do** $R[x]$ Significato: Esegui R per un x scelto random che soddisfi φ Implementa il concetto di non-determinismo

ASMs: riassumendo. . .

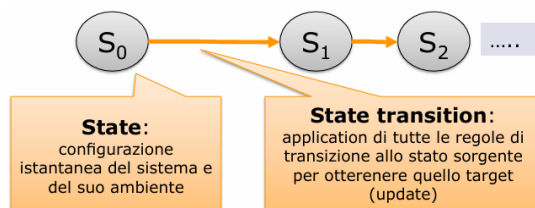
Una Abstract State Machine M è una terna:

- un vocabolario Σ
- uno stato iniziale A per Σ
- un insieme R di nomi di regole con un nome di regola di arità zero, la cosiddetta main rule (l' "entry point" per l'esecuzione della macchina) e una definizione di regola per ogni nome di regola

La semantica delle regole di transizione è data dall'insieme di tutti gli aggiornamenti.

Eeguire una ASM

La run(o esecuzione) di una ASM M è definita come Una sequenza finita o infinita $S_0, S_1, \dots, S_n$ di stati di M , dove S_0 è lo stato iniziale e ciascun stato S_{n+1} è ottenuto dal precedente S_n eseguendo simultaneamente regole di transizione che sono eseguibili in S_n



n b b d n